MODUL PRAKTIKUM PEMROGRAMAN WEB

MODUL 4: DASAR LOGIKA PEMROGRAMAN DENGAN JAVASCRIPT

Fondasi Logika Pemrograman, Manipulasi Data Modern (ES6+), dan Integrasi Antarmuka Web Dinamis

BAB 1: PENGENALAN & FONDASI JAVASCRIPT

1.1 Apa itu JavaScript & Perannya dalam Web Modern

JavaScript adalah bahasa pemrograman tingkat tinggi, dinamis, weakly-typed, dan multi-paradigma yang menjadi standar de facto dalam pengembangan web di seluruh dunia. Bersama dengan HTML dan CSS, JavaScript membentuk segitiga pilar utama web (Web Trifecta):

- **HTML (HyperText Markup Language):** Menyediakan fondasi struktural dan semantik konten pada halaman web.
- **CSS (Cascading Style Sheets):** Mengatur presentasi visual, tata letak, tipografi, warna, dan estetika responsif.
- **JavaScript:** Memberikan 'otak', interaktivitas dinamis, kendali logika, manipulasi dokumen secara real-time, dan komunikasi jaringan asinkron (AJAX/Fetch API).

Awalnya, JavaScript diciptakan pada tahun 1995 oleh Brendan Eich di Netscape Communications sebagai skrip sisi klien (client-side) di peramban. Namun, standarisasi resmi di bawah ECMAScript (terutama lompatan besar pada ECMAScript 2015 atau ES6) telah mengubah JavaScript menjadi bahasa yang sangat matang, efisien, dan serbaguna.

1.2 Di Mana JavaScript Berjalan?

JavaScript tidak hanya hidup di dalam browser. Kode JavaScript memerlukan JavaScript Engine untuk dieksekusi. Terdapat dua lingkungan eksekusi utama:

- **Client-Side (Peramban / Browser):** Setiap browser modern memiliki engine bawaan (contoh: V8 di Chrome/Edge, SpiderMonkey di Firefox, JavaScriptCore di Safari). Di lingkungan ini, JavaScript memiliki akses ke objek peramban seperti window, document (DOM), dan LocalStorage.
- **Server-Side & Standalone Runtime (Node.js, Deno, Bun):** Engine V8 Chrome diekstraksi ke luar browser untuk menciptakan runtime Node.js. Di lingkungan ini, JavaScript berjalan langsung di terminal/server, membaca berkas di hard drive, mengelola database, dan membuat web server mandiri tanpa memerlukan file HTML.

> 📌 **Prinsip Pembelajaran Logika:** Sebelum menggabungkan kode ke dalam peramban dan memanipulasi elemen visual (DOM), seorang programmer wajib menguasai sintaks murni dan logika komputasi data JavaScript terlebih dahulu melalui konsol atau terminal. Pendekatan ini memisahkan pemahaman logika komputasi murni dari manipulasi antarmuka peramban.

1.3 Deklarasi Variabel Modern: var vs let vs const

Sebelum standarisasi ES6 (2015), satu-satunya cara mendeklarasikan variabel adalah menggunakan kata kunci var. Di era modern, var sudah ditinggalkan karena memiliki cakupan fungsi (function-scoped) yang berisiko menimpa variabel global secara tidak sengaja. ES6 memperkenalkan dua kata kunci pengganti yang memiliki cakupan blok (block-scoped):

Karakteristik	var (Legacy / Usang)	let (Modern Mutable)	const (Modern Constant)
Cakupan (Scope)	Function Scope (atau Global)	Block Scope (terbatas di { })	Block Scope (terbatas di { })
Re-assignable?	Ya, dapat diubah nilainya	Ya, dapat diubah nilainya	Tidak, referensi nilai tetap
Re-declarable?	Ya (berbahaya tertimpa)	Tidak (mencegah duplikasi)	Tidak (mencegah duplikasi)
Hoisting	Di-hoist dengan nilai undefined	Di-hoist dalam Temporal Dead Zone	Di-hoist dalam Temporal Dead Zone
Rekomendasi	Hindari sepenuhnya	Gunakan saat nilai berubah	Gunakan sebagai pilihan default

Contoh perbedaan mendasar dalam kode program:

```
// 1. Penggunaan const: Nilai referensi tidak dapat diubah kembali
const namaAplikasi = "Portofolio Digital";
// namaAplikasi = "Aplikasi Lain"; // Error: Assignment to constant variable.

// Catatan: Properti objek atau elemen array pada const tetap bisa dimutasi!
const profil = { nama: "Budi", role: "Developer" };
profil.role = "Lead Developer"; // Valid dan diperbolehkan!

// 2. Penggunaan let: Digunakan untuk nilai yang akan berubah (mutable)
let counter = 0;
counter += 1; // Valid

// 3. Block Scope pada let dan const
if (true) {
    let pesanRahasia = "Hanya terlihat di dalam kurung kurawal ini";
}
// console.log(pesanRahasia); // Error: pesanRahasia is not defined
```

1.4 Tipe Data Dasar & Template Literals

JavaScript mengelompokkan tipe data menjadi dua kategori besar: Tipe Primitif (immutable, disimpan berdasarkan nilai) dan Tipe Non-Primitif / Reference (disimpan berdasarkan referensi memori):

- **String:** Teks yang diapit oleh petik tunggal ('...'), petik ganda ("..."), atau backtick (`...`).
- **Number:** Bilangan bulat maupun desimal (64-bit float), serta nilai khusus seperti NaN dan Infinity.
- **Boolean:** Bernilai logis true atau false.
- **Null:** Menandakan ketiadaan nilai yang sengaja dikosongkan oleh pengembang.
- **Undefined:** Nilai bawaan dari variabel yang dideklarasikan namun belum diisi.
- **Object & Array:** Struktur data non-primitif untuk menyimpan kumpulan data terstruktur.

ES6 memperkenalkan Template Literals menggunakan karakter backtick (`...`) yang mempermudah penggabungan string dan interpolasi ekspresi dinamis (${...}) tanpa operator (+):

```
const nama = "Ahmad";
const jam = 14;
// Sintaks modern (Template Literal):
const pesanModern = `Halo ${nama}, sekarang pukul ${jam}:00 WIB.`;
console.log(pesanModern);
```

1.5 Operator Logika, Perbandingan Strict (===), dan Percabangan

Dalam penulisan logika JavaScript, salah satu kesalahan paling fatal adalah tertukar antara perbandingan longgar (==) dan perbandingan ketat (===):

- **Loose Equality (==):** Membandingkan dua nilai setelah melakukan konversi tipe data otomatis (type coercion). Contoh: 5 == '5' bernilai true. Hal ini berpotensi menimbulkan bug tersembunyi.
- **Strict Equality (===):** Membandingkan nilai sekaligus tipe data secara ketat. Contoh: 5 === '5' bernilai false (number tidak sama dengan string). Standar industri modern mewajibkan penggunaan strict equality (=== dan !==).

JavaScript juga mendukung percabangan ringkas menggunakan Ternary Operator:

```
const nilaiUjian = 85;
// Menggunakan Ternary Operator (kondisi ? jika_true : jika_false):
const statusRingkas = (nilaiUjian >= 75) ? "Lulus" : "Remedial";
console.log(`Status Mahasiswa: ${statusRingkas}`);
```

1.6 Fungsi di JavaScript: Regular Function vs Arrow Function

Fungsi adalah blok kode modular yang dapat dipanggil berulang kali. Pada ES6+, Arrow Function (() => {}) diperkenalkan sebagai alternatif modern:

- **Sintaks Lebih Ringkas:** Menghilangkan kata kunci 'function' dan dapat menghilangkan tanda kurung kurawal serta 'return' untuk ekspresi satu baris (implicit return).

- **Pengikatan Konteks 'this':** Arrow function mewarisi nilai 'this' dari cakupan leksikal sekitarnya (lexical scoping).

```
// Deklarasi fungsi konvensional:
function hitungPajak(nominal, persen) {
   return nominal * (persen / 100);
}

// Ditulis ulang dengan Arrow Function:
const hitungPajakArrow = (nominal, persen) => nominal * (persen / 100);
console.log(hitungPajakArrow(1000000, 11)); // Output: 110000
```

1.7 Struktur Data Koleksi: Array of Objects

Dalam aplikasi web dunia nyata, data yang diterima dari server (API) hampir selalu berbentuk kumpulan objek di dalam array (Array of Objects):

```
const projects = [
   {
      id: 1,
      title: "Sistem Inventaris Keuangan",
      category: "Web App",
      budget: 2500000,
      completed: true
   },
   {
      id: 2,
      title: "Sistem Kasir Digital POS",
      category: "Desktop App",
      budget: 5000000,
      completed: true
   }
];
console.log(projects[0].title); // Output: "Sistem Inventaris Keuangan"
```

1.8 Higher-Order Array Methods (ES6+)

ES6 menyediakan metode bawaan array yang menerima fungsi sebagai argumen untuk menggantikan perulangan manual 'for' loop konvensional:

Metode Array	Tujuan Utama	Nilai Kembalian (Return)	Kapan Digunakan?
.forEach()	Iterasi setiap elemen untuk efek samping	undefined (tidak ada)	Mencetak log, mengirim event
.map()	Transformasi setiap item ke format baru	Array baru (ukuran sama)	Membuat list elemen HTML, format data
.filter()	Menyaring item berdasarkan kondisi	Array baru (ukuran <= asal)	Pencarian, filter status/kategori

Metode Array	Tujuan Utama	Nilai Kembalian (Return)	Kapan Digunakan?
.reduce()	Mengakumulasikan item menjadi 1 nilai	Nilai tunggal (angka/objek)	Menghitung total nominal, agregasi
.find()	Mencari item pertama yang cocok	Satu elemen atau undefined	Mencari detail item berdasarkan ID

```
// 1. filter(): Mengambil proyek yang sudah selesai
const proyekSelesai = projects.filter(item => item.completed === true);

// 2. map(): Mengambil daftar judul proyek dalam bentuk array string
const daftarJudul = projects.map(item => item.title);

// 3. reduce(): Menghitung total seluruh anggaran proyek
const totalAnggaran = projects.reduce((total, item) => total + item.budget, 0);
console.log(`Total Budget: Rp ${totalAnggaran.toLocaleString('id-ID')}`);
```

1.9 Pengantar Document Object Model (DOM)

Setelah logika komputasi data dikuasai, barulah JavaScript dihubungkan ke antarmuka web melalui Document Object Model (DOM), yaitu representasi pohon objek dari dokumen HTML di memori peramban:

- **Seleksi Elemen:** document.getElementById('id-elemen') atau document.querySelector('.kelas-elemen').
- **Manipulasi Konten:** elemen.innerHTML = ' Konten ' atau elemen.textContent = 'Teks Murni'.
- **Event Listener:** Menangkap interaksi pengguna (klik, input, submit) dengan elemen.addEventListener('click', handler).
- **Siklus Hidup DOMContentLoaded:** Memastikan dokumen HTML telah selesai dimuat sebelum manipulasi DOM dijalankan, mencegah error 'Cannot read property of null'.

BAB 2: PANDUAN PRAKTIK TERSTRUKTUR

Praktik dibagi menjadi 2 tahap pembelajaran bertingkat: Tahap 1 berfokus pada penguasaan logika data murni melalui konsol/terminal, dan Tahap 2 berfokus pada integrasi visual ke halaman web menggunakan DOM.

Tahap 1: Eksplorasi Logika JavaScript di Console / Terminal (Tanpa Web)

Pada tahap pertama ini, mahasiswa TIDAK membuka peramban web terlebih dahulu. Seluruh kode dijalankan langsung menggunakan runtime Node.js di terminal Visual Studio Code.

Langkah 1.1: Mempersiapkan Folder & Workspace

1. Buka Visual Studio Code.

2. Buka folder proyek Anda, lalu buat folder baru bernama 'modul-4'/’pertemuan-4’.

3. Buka terminal terintegrasi di VS Code (ctrl + j)

4. Pastikan direktori terminal berada di folder modul-4 dengan perintah: cd modul-4.

Langkah 1.2: Praktik Logika Dasar (dasar_logika.js)

Buat berkas baru bernama dasar_logika.js di dalam folder modul-4, lalu tuliskan kode berikut:

```
// File: dasar_logika.js
// Praktik Dasar Sintaks, Tipe Data, dan Arrow Function

const namaProgrammer = "Budi Santoso";
const tahunMulai = 2022;
const tahunSekarang = new Date().getFullYear();

// Arrow Function menghitung masa pengalaman
const hitungPengalaman = (mulai, sekarang) => sekarang - mulai;
const masaKerja = hitungPengalaman(tahunMulai, tahunSekarang);

// Evaluasi kondisi dengan Ternary Operator
const statusSenioritas = masaKerja >= 2 ? "Mid-Level Developer" : "Junior Developer";

console.log("=== INFORMASI PENGEMBANG ===");
console.log(`Nama        : ${namaProgrammer}`);
console.log(`Pengalaman  : ${masaKerja} tahun`);
console.log(`Kategori    : ${statusSenioritas}`);

// Menguji Strict Equality vs Loose Equality
console.log("\n=== UJI PERBANDINGAN TIPE DATA ===");
console.log("10 == '10'  :", 10 == '10');   // true (type coercion)
console.log("10 === '10' :", 10 === '10');  // false (strict equality)
console.log("Tipe 10     :", typeof 10);
console.log("Tipe '10'   :", typeof '10');
```

Jalankan berkas di terminal dengan mengetikkan perintah:

```
node dasar_logika.js
```

Langkah 1.3: Pengolahan Koleksi Data Proyek (olah_data.js)

Buat berkas baru bernama olah_data.js di dalam folder modul-4 untuk mengolah dataset portofolio menggunakan higher-order functions:

```
// File: olah_data.js
// Praktik Manipulasi Array of Objects dengan ES6+

const projects = [
  {
    id: 1,
    title: "Sistem Inventaris Keuangan",
    category: "Web App",
    budget: 2500000,
    completed: true
  },
```

```
    {
        id: 2,
        title: "Sistem Kasir Digital POS",
        category: "Desktop App",
        budget: 5000000,
        completed: true
    },
    {
        id: 3,
        title: "Landing Page Profil Perusahaan",
        category: "Web App",
        budget: 1500000,
        completed: false
    },
    {
        id: 4,
        title: "Aplikasi Presensi GPS Mobile",
        category: "Mobile App",
        budget: 4500000,
        completed: false
    }
];

// Helper function format Rupiah
const formatRupiah = (angka) => {
    return new Intl.NumberFormat('id-ID', {
        style: 'currency',
        currency: 'IDR',
        maximumFractionDigits: 0
    }).format(angka);
};

console.log("=== 1. SEMUA DAFTAR PROYEK (forEach) ===");
projects.forEach((proj, idx) => {
    const status = proj.completed ? "[SELESAI]" : "[PROSES]";
    console.log(`${idx + 1}. ${status} ${proj.title} (${proj.category}) - ${formatRupiah(proj.budget)}`);
});

console.log("\n=== 2. PROYEK KATEGORI WEB APP (filter) ===");
const webProjects = projects.filter(proj => proj.category === "Web App");
console.log(webProjects);

console.log("\n=== 3. RINGKASAN NAMA & ANGGARAN (map) ===");
const ringkasanProyek = projects.map(proj => ({
    nama: proj.title,
    biaya: formatRupiah(proj.budget)
}));
console.table(ringkasanProyek);

console.log("\n=== 4. TOTAL ANGGARAN (reduce) ===");
const totalAnggaran = projects.reduce((total, proj) => total + proj.budget, 0);
console.log(`Total Estimasi Anggaran Portofolio: ${formatRupiah(totalAnggaran)}`);

const totalSelesai = projects
```

```
    .filter(proj => proj.completed)
    .reduce((total, proj) => total + proj.budget, 0);
console.log(`Total Anggaran Proyek Selesai: ${formatRupiah(totalSelesai)}`);
```

Jalankan berkas di terminal dengan mengetikkan:

```
node olah_data.js
```

Tahap 2: Integrasi Logika ke Halaman Web

Sekarang kita menghubungkan data tersebut ke elemen visual antarmuka web (HTML & CSS).

Langkah 2.1: Membuat File Antarmuka (index.html)

Buat berkas baru bernama index.html di dalam folder modul-4, lalu tuliskan kode HTML semantik berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard Portofolio Proyek - Modul 4</title>
    <style>
        * { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', Tahoma, Geneva, sans-serif; }
        body { background-color: #f1f5f9; color: #0f172a; padding: 30px 15px; }
        .container { max-width: 800px; margin: 0 auto; background: #ffffff; border-radius: 12px; padding: 25px;
box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1); }
        h1 { color: #1e3a8a; font-size: 24px; margin-bottom: 5px; }
        p.subtitle { color: #64748b; margin-bottom: 20px; font-size: 14px; }
        .stats-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 15px;
margin-bottom: 25px; }
        .stat-card { background: #f8fafc; border: 1px solid #e2e8f0; border-radius: 8px; padding: 15px; text-
align: center; }
        .stat-card h3 { font-size: 13px; color: #64748b; text-transform: uppercase; margin-bottom: 5px; }
        .stat-card p { font-size: 20px; font-weight: bold; color: #1e3a8a; }
        .filter-group { display: flex; gap: 8px; margin-bottom: 20px; flex-wrap: wrap; }
        .btn-filter { padding: 8px 16px; border: 1px solid #cbd5e1; background: #ffffff; border-radius: 6px;
cursor: pointer; font-size: 13px; font-weight: 500; transition: all 0.2s; }
        .btn-filter.active, .btn-filter:hover { background: #1e3a8a; color: #ffffff; border-color: #1e3a8a; }
        .project-card { border: 1px solid #e2e8f0; border-radius: 8px; padding: 16px; margin-bottom: 12px;
display: flex; justify-content: space-between; align-items: center; background: #ffffff; transition: transform
0.15s ease; }
        .project-card:hover { transform: translateY(-2px); box-shadow: 0 4px 6px rgba(0,0,0,0.05); }
        .project-info h4 { font-size: 16px; color: #0f172a; margin-bottom: 4px; }
        .project-category { display: inline-block; font-size: 12px; color: #0284c7; background: #e0f2fe; padding:
2px 8px; border-radius: 4px; margin-bottom: 6px; font-weight: 600; }
        .project-budget { font-size: 14px; color: #334155; font-weight: 600; }
        .badge { padding: 4px 10px; border-radius: 20px; font-size: 12px; font-weight: 600; }
        .badge-success { background: #dcfce7; color: #166534; }
        .badge-pending { background: #fef9c3; color: #854d0e; }
    </style>
</head>
<body>
    <div class="container">
```

```
    <h1>Portofolio Proyek</h1>
    <p class="subtitle">Modul 4: Pengolahan Logika Data & Interaktivitas DOM</p>

    <!-- Kartu Ringkasan Statistik -->
    <div class="stats-grid">
      <div class="stat-card">
        <h3>Total Proyek</h3>
        <p id="stat-total-projects">0</p>
      </div>
      <div class="stat-card">
        <h3>Total Anggaran</h3>
        <p id="stat-total-budget">Rp 0</p>
      </div>
      <div class="stat-card">
        <h3>Selesai Dikerjakan</h3>
        <p id="stat-completed-projects">0</p>
      </div>
    </div>

    <!-- Tombol Filter Kategori -->
    <div class="filter-group">
      <button class="btn-filter active" data-category="Semua">Semua Proyek</button>
      <button class="btn-filter" data-category="Web App">Web App</button>
      <button class="btn-filter" data-category="Desktop App">Desktop App</button>
      <button class="btn-filter" data-category="Mobile App">Mobile App</button>
    </div>

    <!-- Kontainer Daftar Proyek -->
    <div id="project-list"></div>
  </div>

  <script src="main.js"></script>
</body>
</html>
```

Langkah 2.2: Menulis Skrip Interaktif DOM (main.js)

Buat berkas baru bernama main.js di dalam folder modul-4, lalu tuliskan kode JavaScript berikut:

```
// File: main.js
// Integrasi Logika Data dan Render ke Elemen DOM

const projects = [
  {
    id: 1,
    title: "Sistem Inventaris Keuangan",
    category: "Web App",
    budget: 2500000,
    completed: true
  },
  {
    id: 2,
    title: "Sistem Kasir Digital POS",
```

```
        category: "Desktop App",
        budget: 5000000,
        completed: true
    },
    {
        id: 3,
        title: "Landing Page Profil Perusahaan",
        category: "Web App",
        budget: 1500000,
        completed: false
    },
    {
        id: 4,
        title: "Aplikasi Presensi GPS Mobile",
        category: "Mobile App",
        budget: 4500000,
        completed: false
    }
];

const formatRupiah = (angka) => {
    return new Intl.NumberFormat('id-ID', {
        style: 'currency',
        currency: 'IDR',
        maximumFractionDigits: 0
    }).format(angka);
};

const updateStats = (dataList) => {
    const totalProjects = dataList.length;
    const totalBudget = dataList.reduce((acc, curr) => acc + curr.budget, 0);
    const completedCount = dataList.filter(item => item.completed).length;

    document.getElementById("stat-total-projects").textContent = totalProjects;
    document.getElementById("stat-total-budget").textContent = formatRupiah(totalBudget);
    document.getElementById("stat-completed-projects").textContent = completedCount;
};

const renderProjects = (filteredList) => {
    const container = document.getElementById("project-list");
    if (!container) return;

    if (filteredList.length === 0) {
        container.innerHTML = "<p style='text-align: center; color: #64748b; padding: 20px;'>Tidak ada proyek
pada kategori ini.</p>";
        return;
    }

    const htmlString = filteredList.map(proj => {
        const badgeClass = proj.completed ? "badge-success" : "badge-pending";
        const statusText = proj.completed ? "Selesai" : "Dalam Pengerjaan";

        return `
            <div class="project-card">
```

```
            <div class="project-info">
                <span class="project-category">${proj.category}</span>
                <h4>${proj.title}</h4>
                <p class="project-budget">${formatRupiah(proj.budget)}</p>
            </div>
            <div>
                <span class="badge ${badgeClass}">${statusText}</span>
            </div>
        </div>
    `;
  }).join("");

  container.innerHTML = htmlString;
};

document.addEventListener("DOMContentLoaded", () => {
  renderProjects(projects);
  updateStats(projects);

  const filterButtons = document.querySelectorAll(".btn-filter");
  filterButtons.forEach(btn => {
    btn.addEventListener("click", (e) => {
      filterButtons.forEach(b => b.classList.remove("active"));
      e.target.classList.add("active");

      const selectedCategory = e.target.getAttribute("data-category");
      if (selectedCategory === "Semua") {
        renderProjects(projects);
      } else {
        const filtered = projects.filter(p => p.category === selectedCategory);
        renderProjects(filtered);
      }
    });
  });
});
```

Langkah 2.3: Uji Coba di Peramban Web

1. Buka berkas index.html di Google Chrome atau peramban pilihan Anda.

2. Buka Developer Tools dengan menekan tombol F12, lalu pilih tab 'Console' untuk memastikan tidak ada pesan error.

3. Uji klik tombol filter 'Semua Proyek', 'Web App', 'Desktop App', dan 'Mobile App' untuk memverifikasi daftar proyek dan status dinamis.

Tahap 3: Alur Version Control (Git & GitHub)

Setelah kode latihan selesai diuji dan berjalan sempurna, simpan seluruh kemajuan ke sistem pelacak versi Git ke branch baru:

```
git status
git add .
git commit -m "Menyelesaikan praktik modul 4 logika javascript & integrasi DOM"
```

```
git branch -M pertemuan-4
git push -u origin pertemuan-4
```

BAB 3: TUGAS MANDIRI

> 📌 **Judul Tugas Proyek:** Pembangunan 'Interactive Portfolio Tracker' dengan Formulir Input Dinamis, Pencarian Real-Time, Ringkasan Finansial Otomatis, dan Aksi Interaktif Kartu.

3.1 Spesifikasi Fitur yang Wajib Dibuat

Kembangkan aplikasi portofolio Anda dari latihan sebelumnya dengan mengimplementasikan 4 (empat) fitur utama berikut:

- **Fitur 1 - Formulir Input Proyek Dinamis (Form Handling & Validation):** Tambahkan form HTML di atas halaman dengan input: Judul Proyek (text), Kategori (select dropdown: Web App, Mobile App, Desktop App, UI/UX Design), Estimasi Anggaran (number), dan Checkbox Status 'Sudah Selesai'. Tangani event 'submit' pada form, cegah reload halaman dengan e.preventDefault(), lakukan validasi (judul tidak boleh kosong dan anggaran harus lebih besar dari 0), buat objek proyek baru dengan id unik (Date.now()), masukkan ke dalam array menggunakan method array atau spread operator, lalu render ulang tampilan daftar proyek dan kartu statistik.
- **Fitur 2 - Pencarian Real-Time & Filter Terpadu (Live Search):** Tambahkan input pencarian teks. Setiap kali pengguna mengetik karakter (event 'input'), saring daftar proyek secara real-time berdasarkan kecocokan nama proyek menggunakan kombinasi .filter() dan .toLowerCase().includes().
- **Fitur 3 - Aksi Interaktif Kartu Proyek (Toggle Status & Hapus Data):** Pada setiap kartu proyek yang di-render, sediakan 2 tombol aksi interaktif: (a) Tombol 'Ubah Status': Mengubah status proyek dari 'Dalam Pengerjaan' menjadi 'Selesai' atau sebaliknya, dan (b) Tombol 'Hapus Proyek': Menghapus item dari array berdasarkan id-nya. Setiap aksi harus secara instan memperbarui tampilan kartu dan memperbarui angka pada kartu statistik.

3.2 Panduan Pengujian Mandiri & Instruksi Pengumpulan

Sebelum mengumpulkan tugas, pastikan Anda telah melakukan checklist pengujian mandiri berikut:

- **Checklist 1:** Buka Console peramban (F12) dan pastikan tidak ada pesan error berwarna merah selama seluruh aksi dijalankan.
- **Checklist 2:** Coba tambahkan minimal 2 data proyek baru melalui form HTML dan pastikan langsung muncul di daftar.
- **Checklist 3:** Coba ketikkan kata kunci di kolom pencarian untuk memastikan filter teks berfungsi.
- **Checklist 4:** Coba klik tombol 'Ubah Status' dan 'Hapus' pada salah satu kartu dan amati apakah kartu berhasil di perbarui/dihapus.

Setelah pengujian selesai, jalankan perintah Git berikut untuk melakukan commit dan push ke repositori GitHub Anda ke branch pertemuan-4 yang sudah dibuat:

```
git status
git add .
git commit -m "Menyelesaikan tugas manajemen portofolio interaktif"
git push -u origin pertemuan-4
```